SOKOINE UNIVERSITY OF AGRICULTURE

School of Engineering and Technology

Course: ABE 721 (AGE 721) - Agricultural Systems Modelling and Simulation

TEST 2 - OFFICIAL MARKING SCHEME

Instructor: Kadeghe Fue, PhD, P.Eng(T)

Examiner's Note:
Marks enclosed in brackets [X Marks] indicate the point allocation for each logical step. Allow for mathematical variations in derived equations as long as the underlying physical/biological principles are sound. For Python code, penalize severe logical flaws, but do not penalize minor syntax errors (like a missing colon) since the code is hand-written.

Section A: System Conceptualization and Mathematical Formulation (35 Marks)

Question 1: Multi-Paradigm Modelling of an Aquaponics Facility (15 Marks)

a) **System Boundary & Spatial Domain (4 Marks):**

- The boundary strictly encompasses the water volume within the hydroponic bed and the crop biomass. [1 Mark]
- **Justification:** A Lumped (ODE) approach is required/justified. [1 Mark]
- Because the flow rate $\dot{Q}$ is "highly turbulent and rapidly mixing," the Nitrate concentration is uniform throughout the entire tank volume instantly. [1 Mark] Therefore, there are no spatial gradients ($dN/dz = 0$), rendering a PDE approach mathematically unnecessary and computationally wasteful. [1 Mark]

b) **ODE Derivation (6 Marks):**

- *Nitrate Mass (N):* Must include the exogenous forcing function adding N and a sink term for uptake.
- $\frac{dN}{dt} = U(t) - k \cdot N \cdot B$ [3 Marks for correct structural balance and linear kinetic term]
- *Crop Biomass (B):* Must show growth directly proportional to the uptake of N.
- $\frac{dB}{dt} = Y \cdot (k \cdot N \cdot B)$ (where Y is a yield coefficient/conversion factor). [3 Marks for linking B's growth to the N sink term]

c) **Python Implementation (5 Marks):**

```
def aquaponics_derivatives(state, t, params):
    N, B = state                        # State unpacking [1 Mark]
    U_t = params['U_t']                 # Parameter extraction [1 Mark]
    k = params['k']
    Y = params['Y']
```

```
    dN_dt = U_t - k * N * B              # N derivative logic [1 Mark]
    dB_dt = Y * k * N * B                # B derivative logic [1 Mark]

    return [dN_dt, dB_dt]                # Correct return format [1 Mark]
```

Question 2: Spatial Discretization in Thermal Soil Sterilization (20 Marks)

a) **Explicit FDM Scheme (6 Marks):**

- Substituting finite differences into the PDE: $\frac{T_i^{n+1}-T_i^n}{\Delta t} = \alpha \frac{T_{i+1}^n - 2T_i^n + T_{i-1}^n}{(\Delta z)^2} + S_i^n$ [3 Marks]
- Isolating the future state: $T_i^{n+1} = T_i^n + \frac{\alpha \Delta t}{(\Delta z)^2}\left(T_{i+1}^n - 2T_i^n + T_{i-1}^n\right) + S_i^n \Delta t$ [3 Marks]

b) **CFL Analysis (6 Marks):**

- State the CFL condition: $\frac{\alpha \Delta t}{(\Delta z)^2} \leq 0.5$ [2 Marks]
- Mathematical impact: If Δz is halved (divided by 2), the denominator $(\Delta z)^2$ is quartered (divided by 4). [2 Marks]
- Conclusion: To maintain stability, the time step Δt must also be quartered. This quadruples the number of temporal iterations, combined with doubling the spatial nodes, leading to an $8\times$ catastrophic increase in computational time. [2 Marks]

c) **Python Implementation (8 Marks):**

```
import numpy as np

def explicit_fdm_step(T_current, alpha, dz, dt, S_array):
    # Initialize the next time step array [1 Mark]
    T_next = np.copy(T_current)

    # Calculate the CFL multiplier [1 Mark]
    multiplier = (alpha * dt) / (dz**2)

    # Vectorized calculation avoiding loops (Loop is also acceptable)
    # [4 Marks for correct spatial slicing or loop indexing]
    T_next[1:-1] = T_current[1:-1] + multiplier * (
                   T_current[2:] - 2*T_current[1:-1] + T_current[:-2]
                   ) + S_array[1:-1] * dt # [1 Mark for source term addition]

    return T_next # [1 Mark]
```

Section B: Numerical Execution and Stochastic Calibration (35 Marks)

Question 3: Metaheuristics in Subsurface Drip Calibration (20 Marks)

a) **LMA vs. PSO (5 Marks):**

- **LMA Failure:** LMA relies on mathematical derivatives (gradients) to move "downhill". On a highly rugged surface (due to air pocket noise), there are many false "valleys" where the local gradient is zero. LMA gets permanently trapped in these local minima. [2.5 Marks]

- **PSO Immunity:** PSO does not use gradients. It uses a "swarm" of particles distributed globally, moving based on stochastic probabilities and the collective memory of the group's "global best", allowing it to jump out of rugged local traps. [2.5 Marks]

b) **Custom Cost Function Derivation (7 Marks):**

- Must present a piecewise, continuous, or custom function $J(p)$ that heavily asymmetricizes the penalty.
- Example: $J = \sum_{i=1}^{n} E_i$, where $E_i = (P_{sim} - P_{obs})^2$ if $P_{sim} \leq P_{obs}$ [3 Marks for under-prediction penalty]
- And $E_i = e^{k(P_{sim} - P_{obs})} - 1$ if $P_{sim} > P_{obs}$ (or a heavily weighted quadratic multiplier like $100 \times (P_{sim} - P_{obs})^2$). [4 Marks for valid over-prediction exponential/heavy penalty]

c) **Python Implementation (8 Marks):**

```
def asymmetric_pressure_cost(P_sim, P_obs):
    total_cost = 0.0 # [1 Mark]

    for sim, obs in zip(P_sim, P_obs): # [2 Marks for iteration]
        error = sim - obs
        if error > 0: # Over-predicting pressure (Catastrophic)
            # Apply exponential penalty [3 Marks for logic]
            total_cost += np.exp(error) - 1
        else: # Under-predicting (Minor inefficiency)
            # Apply standard squared error [1 Mark]
            total_cost += error**2

    return total_cost # [1 Mark]
```

Question 4: Stochastic Risk in Autonomous Harvesting (15 Marks)

a) **Stochastic Resolution (5 Marks):**

- Deterministic models use the "average" density, mathematically erasing the existence of dense patches that cause jams. [2.5 Marks]
- A Log-Normal distribution features a long, heavy right tail representing those rare, extreme high-density patches. Monte Carlo sampling forces the simulation to encounter these mathematically, thereby correctly predicting the probability of jams. [2.5 Marks]

b) **LHS vs. Standard Monte Carlo (10 Marks):**

- **Failure of MC:** Standard Monte Carlo relies on pure random chance. To capture a 1-in-1000 event, you might need millions of iterations to ensure enough samples hit that extreme tail, crashing computer memory. [4 Marks]
- **LHS Mechanism:** Latin Hypercube Sampling mathematically divides the cumulative probability distribution ($0.0 \rightarrow 1.0$) into N equal intervals (strata). [3 Marks]
- LHS forces the algorithm to take exactly one sample from every single interval. This guarantees the 99.9th percentile tail is rigorously sampled even with a very small number of iterations N, drastically cutting computational time while proving the rare risk. [3 Marks]

Section C: System Dynamics and Eco-Hydrological Stability (30 Marks)

Question 5: Jacobian Stability in Lake Eutrophication (15 Marks)

a) **CLD Formulation (5 Marks):**

- Links: $P \xrightarrow{+} A$ [1 Mark], $A \xrightarrow{+} Z$ [1 Mark], $Z \xrightarrow{-} A$ [1 Mark].
- **Reinforcing (R) Loop:** The algae dying and releasing P back into the water ($A \xrightarrow{+} P \xrightarrow{+} A$), OR acknowledging $P \xrightarrow{+} A$ drives continuous growth as an R system. [1 Mark]
- **Balancing (B) Loop:** The predator-prey interaction ($A \xrightarrow{+} Z \xrightarrow{-} A$) forms a balancing loop, restricting growth. [1 Mark]

b) **Stability Theorem Proof (10 Marks):**

- **Eigenvalue Analysis:** The given eigenvalues are $\lambda_{1,2} = -0.05 \pm 0.3i$.
- **Real Part:** The real component is negative (-0.05). According to the Stability Theorem, because the real part is strictly negative, the equilibrium point is Locally Asymptotically Stable (an Attractor). [4 Marks]
- **Imaginary Part:** The presence of complex conjugate imaginary parts ($\pm 0.3i$) indicates rotational/oscillatory behavior in the phase plane (a Spiral Sink). [3 Marks]
- **Physical Trajectory:** Yes, the lake will physically recover from the phosphorus dumping. However, the algae and zooplankton will not simply decay smoothly to zero; their populations will oscillate (boom and crash cycles), with the amplitude of the waves dampening over time until equilibrium is reached. [3 Marks]

Question 6: SPAC and the Genetic Engineering Paradox (15 Marks)

a) **Penman-Monteith Physics (8 Marks):**

- **Albedo (α):** A lower albedo means the dark leaves absorb more incoming solar radiation. In the PM equation, this increases the Net Radiation (R_n) term in the numerator. This structurally attempts to **INCREASE** total ET (driven by thermodynamics). [4 Marks]
- **Aerodynamic Resistance (r_a):** A higher r_a (curled leaves trapping air) lowers the efficiency of convective mass transfer. In the PM equation, a higher r_a in the denominator of the aerodynamic term structurally attempts to **DECREASE** total ET. [4 Marks]

b) **Environmental Limit Proof (7 Marks):**

- If the environment is extremely humid and windless, the Vapor Pressure Deficit, $(e_s - e_a)$, approaches exactly zero. [2 Marks]
- Mathematically, this causes the entire aerodynamic mass-transfer term in the numerator of the PM equation $\left(\frac{\rho_a c_p (e_s - e_a)}{r_a}\right)$ to collapse to zero. [2 Marks]
- Therefore, the aerodynamic modification (r_a) is rendered mathematically irrelevant. The equation is dominated entirely by the thermodynamic Radiation term (R_n). [1.5 Marks]
- **Conclusion:** The lower albedo modification dominates, and the GM crop will paradoxically consume **MORE** water than the non-GM crop in this specific humid environment. [1.5 Marks]